

Welcome back! If you found this question useful,
don't forget to vote both the question and the answers up.

[close this message](#)

A* graph search time-complexity

Asked 8 years, 5 months ago Modified 7 years, 3 months ago Viewed 40k times



Some confusion about time-complexity and A*.

13



According to [A* Wiki](#) the time-complexity is exponential in the depth of the solution (shortest path):

The time complexity of A* depends on the heuristic. In the worst case of an unbounded search space, the number of nodes expanded is exponential in the depth of the solution (the shortest path) d : $O(b^d)$, where b is the branching factor (the average number of successors per state).



The comment to [this accepted answer](#) points out that it makes more sense to give the complexity in terms of the size of the graph and therefore it should be $O((|V| + |E|) \cdot \log|V|)$

Obviously, if the heuristic assigns a value of 0 to every node, A* becomes Dijkstra's algorithm and any uniform cost heuristic will essentially disable the heuristic.

If we assume the heuristic to be $O(1)$ (and consistent), it would make sense that the worst case is essentially degrading A* to Dijkstra's algorithm which has complexity

$$O(|E| + |V| \log |V|)$$

with a min-priority queue implementation (Fibonacci heap).

Am I wrong?

Any book etc I have been looking at always gives the complexity in terms of the depth of the solution

[algorithms](#) [algorithm-analysis](#) [graphs](#) [runtime-analysis](#) [shortest-path](#)

[Share](#) [Cite](#) [Edit](#) [Follow](#) [Flag](#)

edited May 23, 2017 at 12:37



asked Apr 19, 2016 at 16:17



2 Answers

Sorted by: [Highest score \(default\)](#)



These are basically two different perspectives or two different ways of viewing the running time. Both are valid (neither is incorrect), but $O(b^d)$ is arguably more useful in the settings that typically arise in AI.

18



In the algorithms community and CS theory community, folks there tend to like to count the running time as a function of the number of vertices and edges in the graph. Why? In that context, worst-case running time is what makes most sense; also, in the problems that are typically considered in that community, in the worst case we need to examine the entire graph, so you typically can't hope to do better than $O(|V| + |E|)$.



In the AI community, folks tend to count the running time differently. They often consider a specific kind of graph: a tree with branching factor b . Also, in the situations that arise there, the graph is often infinite or very large. Typically we try hard to avoid examining all of the graph -- that's often one of the major goals of the algorithms. Thus, counting complexity in terms of $|V|$ and $|E|$ doesn't make sense: $|V|$ may be infinite, and in any case, we don't plan on examining all of the graph, so all that matters is the number of vertices we actually visit, not the number that may exist elsewhere but that we don't care about.

So, for the situations that often arise in the AI community, it's often more meaningful to measure the running time in terms of the branching factor of the tree (b) and the depth of the goal node (d). Typically, once we find the goal node, the algorithm stops. In such a tree, if we examine every vertex at depth $\leq d$ before we find the goal node, we'll end up visiting $O(b^d)$ vertices before we stop. Thus, if you like, you could think of this as visiting a subset of the graph with $|V| = O(b^d)$ (where now V includes only the vertices we visit) and $|E| = O(b^d)$ (E includes only the edges we look at), and you could think of an $O(b^d)$ -time algorithm as one whose running time is $O(|V| + |E|)$... though this is a bit of an abuse of the $|V|, |E|$ notation. Anyway, hopefully you can see why $O(b^d)$ is more informative than $O(|V| + |E|)$ in this context.

[Share](#) [Cite](#) [Edit](#) [Follow](#) [Flag](#)

answered Apr 19, 2016 at 17:48

Welcome back! If you found this question useful,
don't forget to vote both the question and the answers up.

[close this message](#)

- ▲
- 2
- ▼
- It is common in the combinatorial search community to define search spaces *implicitly*, that is, as a set of states and transitions between them - as opposed to *explicitly*, that is, as concrete sets of vertices and edges. In implicit search spaces, states can be represented as vertices and transitions as edges, however, in many cases the practical set of states *may not have finite bounds* and therefore the number of edges and vertices cannot always be defined with finite cardinalities (either practically or theoretically). Thus for many applications it makes more sense to define performance in terms of the branching factor b , as opposed to vertex and edge cardinalities.

🔖 Share Cite Edit Follow Flag



answered Jun 9, 2017 at 15:39

 **thayne**
141 5